Constraint Satisfaction Problem.

A solution to CSP is assignment to each variable such that each constraint is satisfied.

Ex :- n-Queens Problem

Given $\boxed{\mathbb{R} = \langle X, D, C \rangle}$

For a binary CSP $\quad Sol(\mathbb{R})$. $\quad$ (scope, relat

An assignment $\bar{a}$ on scope S $\{x_1, x_2, \ldots\}$ satisfies a constraint C_i, iff $S_i \subseteq S$ and

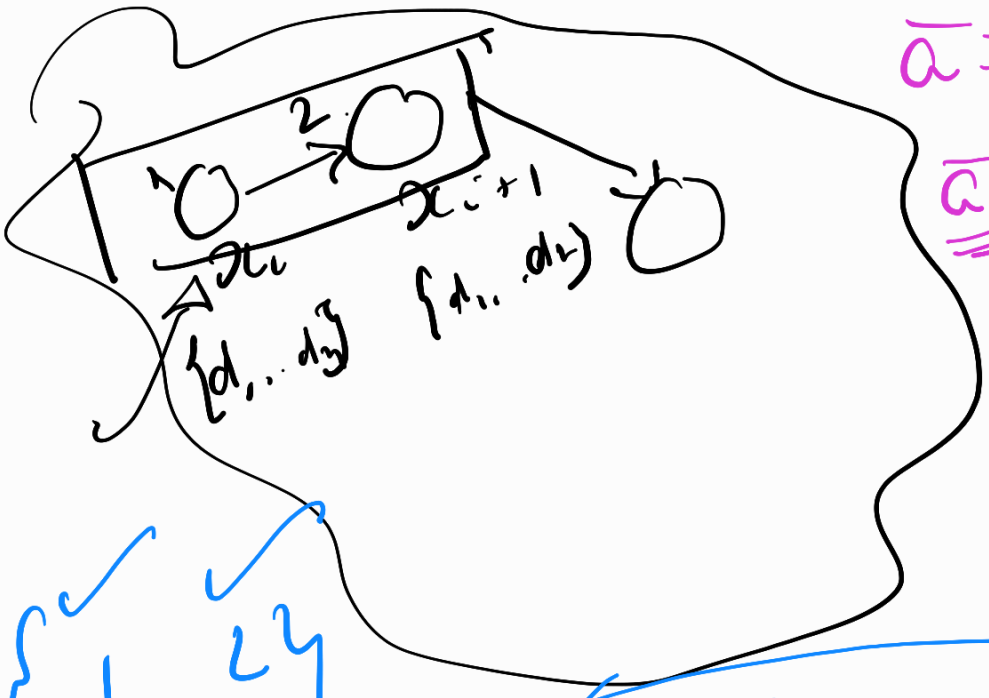
$S_i \subseteq S \{Q_1, Q_2, Q_3, Q_4\}$.

$R_{12} = \{Q_1, Q_2\}$

$a = \Pi_{S_i} \in R_i$

$R \rightarrow \{3, 1, 2, 4\}$.

$R_{12} = \{(3,1)\}$



$\bar{a} = \{3, 1\}$

$\bar{a} = \{3, 1, 2\}$

R_{12} $S_i = \{Q_1, Q_2, Q_3\}$

R_{13}

R_{23}

$\{1, 2\}$

$\{1, 3, 2\}$

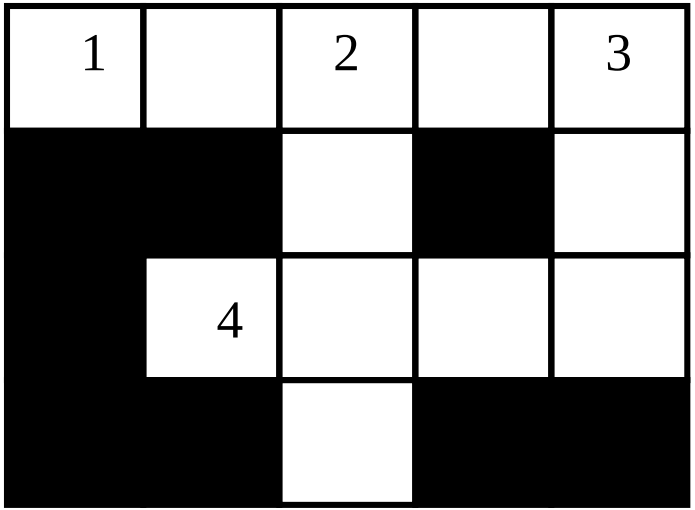
$\bar{a}$ = Satisfies $\bar{a}$

$(K-L)^n$ $\quad L$ $\quad K$

CSP Solution Overview

- CSP Graph Creation:
  - Create a Node for Every Variable. All possible Domain Values are initially Assigned to the Variable
  - Draw edges between Nodes if there is a Binary Constraint. Otherwise Draw a hyper-edge between nodes with constraints involving more than two variables
- Constraint Propagation:
  - Reduce the Valid Domains of Each Variable by Applying Node Consistency, Arc / Edge Consistency, K-Consistency, till no further reduction is possible. If a solution is found or the problem found to have no consistent solution, then terminate
- Search for Solution:
  - Apply Search Algorithms to Find Solutions
  - There are interesting properties of CSP graphs which lead of efficient algorithms in some cases: Trees, Perfect Graphs, Interval Graphs, etc
  - Issues for Search: Backtracking Scheme, Ordering of Children, Forward Checking (Look-Ahead) using Dynamic Constraint Propagation
  - Solving by Converting to Satisfiability (SAT) problems

CSP Graph for Crossword



Word List:

astar, happy, hello, hoses, live, load, loom, peal, peel, save, talk, ant, oak, old

Constraint Propagation Steps

- Constraints
  - Unary Constraints or Node Constraints
  - Binary Constraints or Edges between CSP Nodes
  - Higher order or Hyper-Edges between CSP Nodes
- Node Consistency
  - For every Variable V_i, remove all elements of D_i that do not satisfy the Unary Constraints for the Variable
  - First Step is to reduce the domains using Node Consistency
- Arc Consistency
  - For every element x_ij of D_i, for every edge from V_i to V_j, remove x_ij if it has no consistent value(s) in other domains satisfying the Constraints
  - Continue to iterate using Arc Consistency till no further reduction happens.
- K-Consistency or Path Consistency
  - For every element y_ij of D_i, choose a Path of length L with L variables, use a consistency checking method similar to above to reduce domains if possible

Arc Consistency Algorithm AC-3

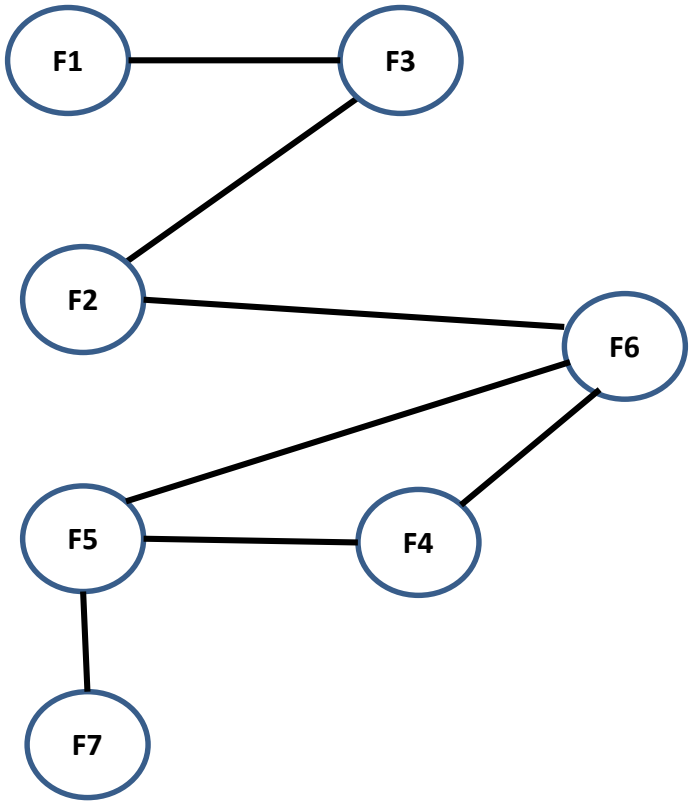
```
AC-3(csp) // inputs - CSP with variables, domains, constraints
    queue ← local variable initialized to all arcs in csp
    while queue is not empty do
        (Xi, Xj) ← pop(queue)
        if Revise(csp, Xi, Xj) then
            if size of Di = 0 then return false
            for each Xk in Xi.neighbors-{Xj} do
                add (Xk, Xi) to queue
    return true
```

```
Revise(csp, Xi, Xj)
    revised ← false
    for each x in Di do
        if no value y in Dj allows (x, y) to satisfy constraint between Xi and Xj then
            delete x from Di
            revised ← true
    return revised
```

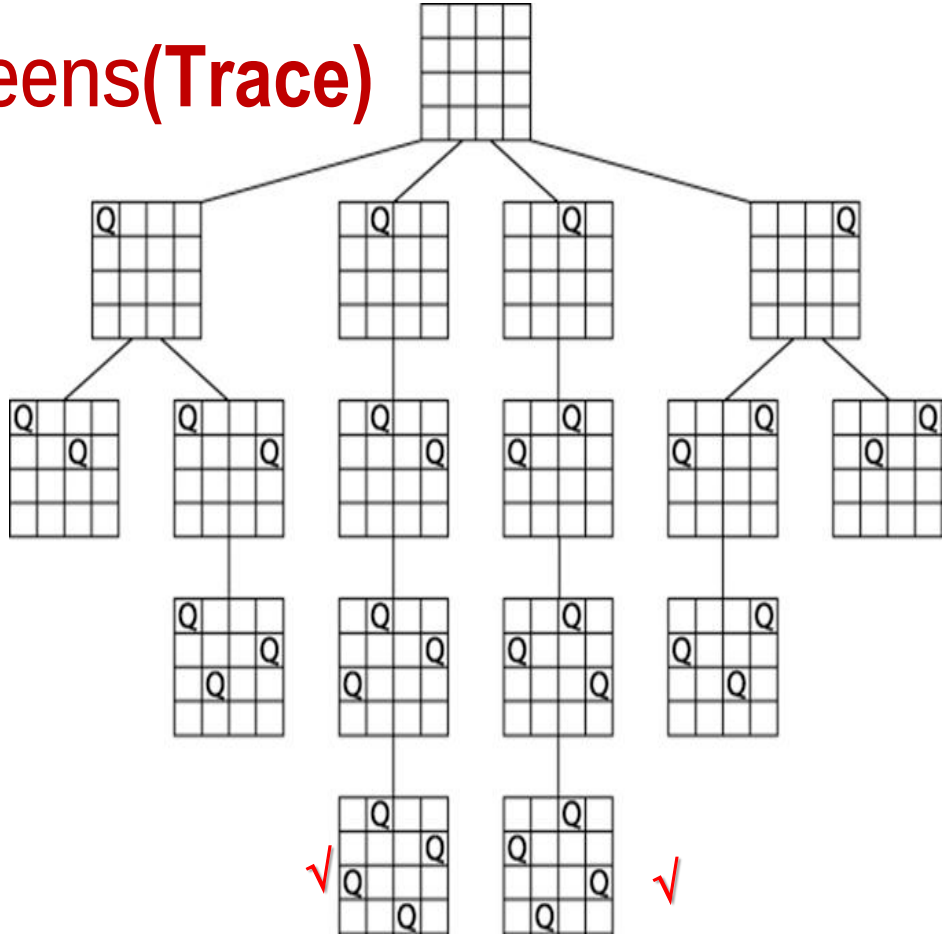
Time complexity: $O(\mathbf{e}d^3)$

Backtracking for Airline Gate Scheduling

Flight No	Dep Time	G Start	G End
F1	7:00	6:15	7:15
F2	8:30	7:45	8:45
F3	7:45	7:00	8:00
F4	9:45	9:00	10:00
F5	10:00	9:15	10:15
F6	9:00	8:15	9:15
F7	11:00	10:15	11:15



Search 4-Queens(**Trace**)



Strategies for CSP Search Algorithms

- Initial Constraint Propagation
- Backtracking Search
  - Variable Ordering
    - Most Constrained Variable / Minimum Remaining Values
    - Most Constraining Variable
  - Value Ordering
    - Least Constraining Value leaving maximum flexibility
  - Dynamic Constraint Propagation Through Forward Checking
    - Preventing useless Search ahead
  - Dependency Directed Backtracking
- SAT Formulations and Solvers
- Optimization
  - Branch-and-Bound
  - SMT Solvers, Constraint Programming
- Learning, Memoizing, etc
- CSP Problems are NP-Hard in General